

# CS 4632 A.I. Heads-up No Limit Texas Hold'em Poker Tournament Simulation

[https://github.com/ian-roundkicker/CS4632\\_A.I.-Heads-up-No-Limit-Texas-Hold-em-Poker-Tournament-Simulation](https://github.com/ian-roundkicker/CS4632_A.I.-Heads-up-No-Limit-Texas-Hold-em-Poker-Tournament-Simulation)

Ian Blanchard

May 4, 2026

## **Abstract**

I created a simulation of Heads-Up No-Limit Texas Hold'em Poker with five different selectable behaviors that compete until one bot runs out of cards. I wrote multiple C++ classes, utilizing Henry R. Lee's poker evaluator library to efficiently determine the better card hand and the SQLite3 amalgamation to collect data over the course of each run. This allows for easy introduction of new A.I behaviors with a fork of the repository. I found from comparing the performance of each behavior that without opponent modeling, always raising and never folding was the best-performing behavior.

# Contents

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                     | <b>4</b>  |
| <b>2</b>  | <b>Background and Literature Review</b> | <b>4</b>  |
| <b>3</b>  | <b>Model Design and Architecture</b>    | <b>6</b>  |
| <b>4</b>  | <b>Implementation</b>                   | <b>7</b>  |
| 4.1       | Agents . . . . .                        | 7         |
| 4.2       | Data Collection . . . . .               | 8         |
| 4.3       | Challenges . . . . .                    | 8         |
| <b>5</b>  | <b>Experimental Setup</b>               | <b>11</b> |
| <b>6</b>  | <b>Results and Analysis</b>             | <b>12</b> |
| <b>7</b>  | <b>Validation and Verification</b>      | <b>16</b> |
| <b>8</b>  | <b>Discussion</b>                       | <b>17</b> |
| <b>9</b>  | <b>Conclusion and Future Work</b>       | <b>17</b> |
| <b>10</b> | <b>Appendix</b>                         | <b>18</b> |

## List of Figures

|   |                                         |   |
|---|-----------------------------------------|---|
| 1 | A rough diagram of components . . . . . | 7 |
| 2 | ERD for the database tables . . . . .   | 8 |

# 1 Introduction

Many months ago I read a story about a Freshman aiming for their Computer Science degree entering an A.I. Poker Contest at their college. Since they were inexperienced in programming, they made their bot "go all in" if it was their turn. Despite their inexperience, this allowed them to "break" most bots because the creators did not anticipate such a move. It is a classic story about an underdog succeeding by taking great risks. Whether or not the story is true is not my concern, it made me interested in the general concept of Poker A.I.

As I was researching poker bots, I learned of the work of the University of Alberta's Computer Poker Research Group. Some years ago, they had apparently "solved" poker by creating a bot that makes the best possible moves at every turn. Unlike other solved games such as Connect Four, Tic-Tac-Toe, or Checkers, Poker is a game of imperfect information, meaning that players do not know the game's current state. I initially wanted to train my own version of the bot using the source code, but I was unable to due to a limitation of file space and difficulty setting up the bot. I also attempted to include a previous bot created by the group, named Loki, but I was unable to find the source code and did not have the time to recreate it from information found in a Master's thesis.

The question I intend to answer is:

- Can I create a bot that can defeat a bot that only raises?
- Can I create a simulation that would allow for implementation of more advanced models in a fork?

I recognize the weakness of these questions. They are more concerned with my own ability rather than something more objective. However, my initial plan to import better Poker bots had failed, leaving me with only this reality. I have performed as well as I could under these restrictions.

# 2 Background and Literature Review

This project is greatly inspired by concepts found in the field of Game Theory, which became a distinct field following the publication of John von Neumann and Oskar Morgenstern's book Theory of Games and Economic Behavior. The book connects card and board games to the field of economics, and reduces the concept of these games to an abstract form. The variant of Poker I intend to emulate, Heads Up (2-player) No Limit (bets aren't fixed) Texas Hold'em (build a five-card hand given two dealt cards and five publicly viewable community cards) Poker, is considered a two-person zero-sum game [1]. This means that the winning player gains whatever the other player lost, or in other words, the combined winnings of both players = 0. Poker is also by definition a game of imperfect information, which requires the use of probability to quantify hidden states to decide optimal play. Theory of Games and Economic Behavior actually

discusses 5-card draw poker in a specific section. The authors ascertain that there is a best strategy for Poker, and that is to always bid high on strong hands, less on weak hands, and irregularly bluff [1]. There is significantly more to the game (and other games/topics) that this book discusses, but for the sake of brevity, I shall apply this to my other sources.

Cepheus is the name of the bot created by the University of Alberta Poker Research Group in 2015. It uses a variant of Counterfactual Regret Minimization, a regret-matching algorithm that generates a probability given an information set. Through repeated sub-optimal play, the algorithm increases a variable called regret, which informs it what it would have been better off doing. This modifies the weight of the probabilities to one that would be better given the information set. Through more optimal, CFR and CFR+ reduce regret. CFR+ differs from CFR in that it (1) combines multiple information sets together to reduce the calculation time and required storage space and (2) resets negative regrets to 0 in order to speed up calculation. The overall goal of this process is to reach a "Nash Equilibrium," an ideal state in game theory (conceptualized years after Theory of Games and Economic Behavior was first published) where the player can't improve their gain. Despite this, creating Cepheus still required 262 TiB of regret values and strategy probabilities and 900 core-years of computation, which required the team to significantly compress the data on disk and recursively run sub-games on different devices. Another important thing to note is that Cepheus was created to run on Head Up Limit Texas Hold'em, which sets bets to a fixed value, preventing a "going all in" situation. This reduces the number of possible actions in a single turn since the magnitude of a raise is no longer in consideration.

Loki was created by the University of Alberta Poker Research Group in 1998 and was created with the goal of implementing opponent modeling into Poker A.I. behavior. Opponent modeling is one of the most significant and difficult aspects of creating powerful A.I players. Without it, the bot must be reduced to predictable (or random) actions, which can easily be exploited as the pattern is revealed through play. Loki weighs its hand's strength, potential to improve, and the opponent's moves to determine what action it should take. It estimates the opponent's current hand based on their current betting action and their history in previous rounds [2].

Even though none of the bots I implemented utilized opponent modeling, another programmer could, if they wished to, implement such a Poker model. To this end my approach uses class inheritance to ensure that the method call for taking a betting action is shared between all bots. Therefore, I just have to give bots a constructor, a method to take a betting action, and a method to read the opponent's cards following each round in order to allow for more advanced behavior than I implemented. The bots I implemented operate under the restriction of no opponent modeling for simplicity and to learn if only raising can still be effective. One of my bots, *Weighted Random*, uses a 2D array to determine the odds that it would fold based on their hole cards' potential to improve (inspired by a component of Loki). Another bot *Cautious*, will use the current 5-card hand to determine whether or not to fold (inspired by another

component of Loki).

### 3 Model Design and Architecture

Texas Hold'em Poker contains the following rounds:

- *Blind* This is a mandatory pre-round bet before players are shown their cards. The dealer posts a small blind as their initial bet, then the other player posts a big blind, which exceeds that of the small blind. This bet can be called/raised by the other players before the first round can commence.
- *Pre-flop* Each player is given two cards from the deck and places bets.
- *Flop* Three cards are added visible for all players to see. Since any or all players can use these cards, they are called community cards. Once these cards are added, players can place bets.
- *Turn* One more community card is added, then players place bets.
- *River* The last community card is added, the players place bets.
- *Showdown* The players use their best 5-card hand using their hole cards and community cards to determine who wins the round and the bet.

For my simulation, I decided to exclude the blind round and replace it with an initial bet of one chip. Despite the importance of the round to Texas Hold'em, it only affects the initial bet, and I would have to create new actions for the various strategies. The round's importance is primarily psychological, with players deciding whether to play in the round with increased starting stakes, which matters to an uncaring algorithm. In addition, the cards are dealt via algorithm and not by "players," a blind round would have to assign the dealer arbitrarily. However, my simulation does define a player 1 and 2, which can be customized with a JSON file.

I decided to separate the following into their own components:

- Card
- Player Hand
- Game Logic
- Player Behavior

Data collection is done with an SQLite database connection. The connection is part of the driver class, and is passed to the game class, which uses it to collect data as it handles game logic.

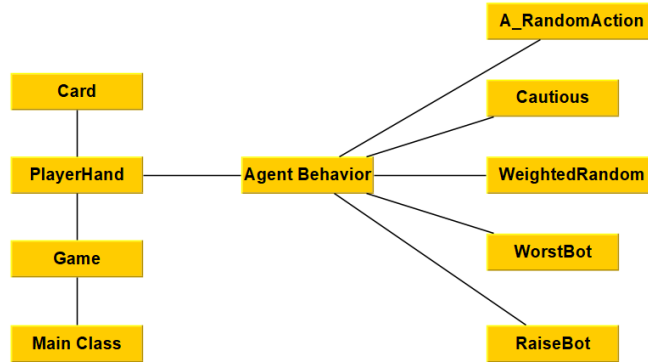


Figure 1: A rough diagram of components

## 4 Implementation

I used the C++ language, the standard library, the SQLite3 amalgamation, and Henry R. Lee’s Poker evaluator to facilitate my program. I use a .json file in the project directory to allow for easy selection of the number of starting chips, the behaviors of each player, and whether to append to or overwrite the current database, and the number of games to simulate before stopping the program.

### 4.1 Agents

These are the basic agents I implemented:

- *A\_RandomAction* has a 1/3 chance to choose one of the three options: fold, call/check, or raise. If it selects raise but has no extra chips, it will automatically call.
- *WeightedRandom* uses a table to determine the odds it will fold based on the rank of the hole cards. This agent makes two other improvements on *A\_RandomAction*: it will not fold if the current bet is greater or equal to the amount the player has, and it takes the odds to the fourth root to account for the function being called four times.
- *RaiseBot* will raise by 1 chips if it has the ability to. *RaiseBot* is a more subtle version of my All\_In bot that I had initially planned. I decided to make it only raise one at a time in order to exploit weaknesses in my models that operate on chance.
- *WorstBot* was made as a joke, simply acting as the worst possible play.
- *Cautious* will fold unless its hand with the first three community cards is in the top 33% of possible hands. At the turn (when the fourth card is

added), it will check if its hand is in the top 25% of possible hands. When the last community card is added, it will check if its hand is in the top 20% of possible hands. I reasoned that a model that would *only play the top  $x\%$  of hands* is not as effective since 5-card hands tend to improve on turn and river rounds.

## 4.2 Data Collection

Due to the dynamic nature of rounds within games and turns within rounds, I decided to use SQLite to store data collected. This is portable since any OS can download DB Browser for SQLite and open the database file within a minute. In my original plan I wanted to add the card order before shuffling. I completely abandoned the idea because it is useless data that has nothing to do with the game. Instead, I am collecting more in-depth data regarding each action, round, and game.

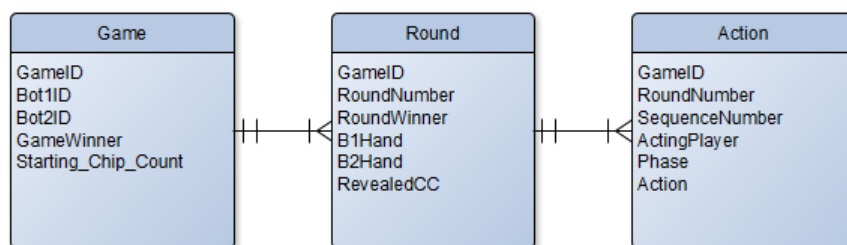


Figure 2: ERD for the database tables

The largest issue with my database is that variables regarding the type of bot, card, phase of the game, or action taken use in-simulation numbers that require some understanding of the simulation's inner workings to understand. Below is a set of tables describing the three database tables:

## 4.3 Challenges

There were many challenges overcome and challenges I failed to overcome. I have had to make a major project in C++, a language I only briefly touched for a few weeks in the first year as a joke. I had made a few programs in C in the semester prior, which I thought would be completely fine since C++ is made to work with existing C code. I had assumed that C++ was just C with classes and some other features. Unfortunately, this assumption is far from the truth. C++ programming standards are not what I expected. I utilized the standard library for simple things like sorting the array of cards. I initially attempted to use an array of integer pointers for the players' hands (at this early stage, I intended to implement 5-card draw poker before I implemented Texas Hold'em). However, my attempts to sort the player's card hand via. Lambda



**Table: Game****Primary Key:** GameID

| Name of Variable    | Datatype      | Description                                                                                                           |
|---------------------|---------------|-----------------------------------------------------------------------------------------------------------------------|
| GameID              | Autoincrement | Allows for an ordered chronology of game order.                                                                       |
| Bot1ID              | Integer       | Identifies the behavior of the Bot. 0 = Random, 1 = Weighted Random, 2 = Always Fold, 3 = Always Raise, 4 = Cautious. |
| Bot2ID              | Integer       | Same functionality as Bot1ID, just for the second agent.                                                              |
| GameWinner          | Integer       | Identifies the winner of the game, 1 for Bot 1, 2 for Bot 2                                                           |
| Starting_Chip_Count | Integer       | The number of chips each play has                                                                                     |

Table 1: Schema definition for the Game table

**Table: Round****Primary Key:** (GameID, RoundID)

| Name of Variable | Datatype    | Description                                                                              |
|------------------|-------------|------------------------------------------------------------------------------------------|
| GameID           | Integer     | Foreign Key referring to Game.                                                           |
| RoundID          | Integer     | Identifies the round number in the game. Round number handled by program and not DBMS.   |
| RoundWinner      | Integer     | Identifies the winner of the round, just like GameWinner from Game.                      |
| B1Hand           | VARCHAR(16) | List of cards in the first bot's hand. Each card is represented by an integer from 0-52. |
| B2Hand           | VARCHAR(16) | Same as B1Hand but for the second bot.                                                   |
| RevealedCC       | VARCHAR(64) | List of cards from the community cards.                                                  |

Table 2: Schema definition for the Round table

**Table: Action**

**Primary Key:** (GameID, RoundID, SequenceNumber, Phase)

| Name of Variable | Datatype | Description                                                                                     |
|------------------|----------|-------------------------------------------------------------------------------------------------|
| GameID           | Integer  | Foreign Key referencing the same variable from Game.                                            |
| RoundNumber      | Integer  | Foreign Key referencing the variable from Round.                                                |
| SequenceNumber   | Integer  | Represents the order of actions within a particular round.                                      |
| ActingPlayer     | Integer  | Identifies whether the first or second player is making the action.                             |
| Phase            | Integer  | Represents the phase of the round: 0 = preflop, 1 = flop, 2 = turn, 3 = river.                  |
| Action           | Integer  | Represents the action taken: -1 = fold, 0 = call/check, 1 or greater = raise by that magnitude. |

Table 3: Schema definition for the Action table

function caused heap corruption, forcing me to change my approach to using a vector from the standard library.

In order to compare the two 5-card hands each player drew and print out the winner, I shifted towards importing the Poker Evaluator library I learned about. I cloned the repository on GitHub and proceeded to attempt to build the project via. Command Prompt. This did not go well because the version of CMake I had came with Strawberry Perl which I had used for MikTeX a couple of years back. I did not have an output when I used these commands:

```
mkdir -p build
cd build
cmake -DBUILD_TESTS=OFF ..
```

For some reason, I decided that because I was not getting anything from this that I would use the other methods which required Google Test suite because this gave me more output than the correct method. I proceeded to improvise by using ninja after the initial commands, which gave me the .a files. However, this was completely useless, as I was using Visual Studio's MSVC for compilation and not GCC. By modifying the cmake command to the following:

```
cmake -G "Visual Studio 17 2022" -A x64 -DBUILD_TESTS=OFF ..
```

I got the following message: “could not find any instance of Visual Studio.” This was quite terrible. I looked on the CMake website and quickly learned that the version of CMake Strawberry Perl used was very much out of date. I was a fool, but I now had an up-to-date version of CMake, allowing me to create .vcxproj files and one .slnx file. I opened the .slnx file in Visual Studio, allowing me to right-click and build the .lib files. I then modified my project’s properties to link the .lib and .h files. I ended up only using the functions from the .h files, but I now had an easy way to check which player had a better hand.

I made a function to rank the hands from least to greatest (best to worst), which required me to write a Lambda function to sort a copy of the vector players. This allows for quickly determining the winner of the round.

Thankfully, switching from 5-card draw to Texas Hold’em was substantially easier, and implementing behaviors wasn’t horrible. Unfortunately, I had to change from a raw pointer to a unique pointer due to some crashed I experienced while implementing the behaviors. This is yet another example of me learning how C++ programming differed from C. Finally, when implementing data collection, I was forced to pass the database connection pointer from the SQLite amalgamation to the Game class’s *playGame* method in order to allow for individual round and action data could be collected.

## 5 Experimental Setup

There are few parameters to customize when making a Poker simulation, as it is ultimately a game about betting with different cards in the same deck.

- **num\_games**: The number of games to run within a single simulation.
- **num\_chips**: The number of chips that each player has at the beginning of the game.
- **append\_to\_db**: Whether to erase the previous database or append to it upon starting the simulation.
- **Bot1**: Which agent/behavior the first player uses.
- **Bot2**: Which agent/behavior the second player uses.

The most important parameters are those affecting the number of chips, and the ones affecting player behavior. The number of chips is important since it affects game length. Most agents will also not fold if they have only one chip since that causes automatic loss.

The Bot1 and Bot2 parameters represent the behavior of the first and second players. These parameters take in any number from 0-4 to decide one of five behaviors:

0. A.RandomAction
1. WeightedRandom

2. WorstBot
3. RaiseBot
4. Cautious

Player 1 will always be the first to call, bet, or raise. Player 2 will therefore always make a decision second.

I ran the simulation a total of 1,000 games for each permutation of bots, giving me a grand total of 20,000 games. In addition, I did a simulation of 10,000 games between *WeightedRandom* and *Cautious*, changing the starting number chips by 1 for every 1,000 games.

As stated before, I collected this data using the SQLite3 amalgamation C library.

## 6 Results and Analysis

I am starting with an analysis of starting chip count. For brevity, I decided to use the matchup that is most likely to have the highest impact. I have determined that this is the match with the highest impact since *Cautious* tends to waste its available chips in rounds and *Weighted Random* performs the closest against it (it is almost a 50/50 match). In addition, turn order has very minor impact on the results of the match (as we will see later). To restate from the previous section, I ran 1,000 games with this matchup with a starting chip count from 1-10. I then created a script to run a statistical significance test to see if the difference in win rate is significant. The wins and win rate being counted are of *Cautious*.

| Starting Chips | Total Games | Wins | Win Rate | CI Lower | CI Upper |
|----------------|-------------|------|----------|----------|----------|
| 1              | 1000        | 490  | 0.490    | 0.459016 | 0.520984 |
| 2              | 1000        | 491  | 0.491    | 0.460015 | 0.521985 |
| 3              | 1000        | 438  | 0.438    | 0.407249 | 0.468751 |
| 4              | 1000        | 428  | 0.428    | 0.397333 | 0.458667 |
| 5              | 1000        | 466  | 0.466    | 0.435081 | 0.496919 |
| 6              | 1000        | 440  | 0.440    | 0.409234 | 0.470766 |
| 7              | 1000        | 420  | 0.420    | 0.389409 | 0.450591 |
| 8              | 1000        | 400  | 0.400    | 0.369636 | 0.430364 |
| 9              | 1000        | 415  | 0.415    | 0.384461 | 0.445539 |
| 10             | 1000        | 447  | 0.447    | 0.416184 | 0.477816 |

Table 4: Win Rate and Confidence Intervals by Starting Chip Count

Despite a general trend of win rates falling as the number of starting chips increase, *Cautious* does have a spike in win rate when starting with 5 and 10 chips respectively. However, it never reaches the win rate it had enjoyed at 1 starting chip, when the game was entirely determined by the starting hand. It seems that *Cautious* loses the game of attrition by folding too often.

| Metric                | Value                                           |
|-----------------------|-------------------------------------------------|
| Chi-Squared Statistic | 34.3445                                         |
| P-Value               | 0.0001                                          |
| Conclusion            | Statistically significant effect ( $p < 0.05$ ) |

Table 5: Chi-Squared Test for Starting Chip Count vs Win Rate

I next compared the impact of turn order. Unfortunately as the table is too long, it is included in the appendix.

1. *Random Action vs. Weighted Random*: *Random Action* won 54 more games when it went second than when it went first.
2. *Random Action vs. Worst Bot*: *Worst Bot* lost every game when it went first, but won 31 out of 1,000 games when it went second. This difference is the result of *Random Action* folding until it is out of chips before *WorstBot* has the ability to do the same.
3. *Random Action vs. Cautious*: *Random Action* won 131 out of 1,000 games when playing first, but then won 249 out of 1,000 rounds when playing second, almost double. This is a similar situation to the previous pairing where one bot's win rate increased mainly due to the other folding before it could fold.
4. *Weighted Random vs. Raise Bot*: Of the games where turn order matters, this is the one where it matters the least. Looking at all the other statistics, it seems that *Weighted Random* performs worse when it is player 2 in general.

To finish this section, here are some less important statistics. I shall derive no confidence intervals from these.

Table 6: Fold Frequency Summary

| Behavior        | Total Actions | Total Folds | Fold Percentage (%) |
|-----------------|---------------|-------------|---------------------|
| Random Action   | 62,617        | 20,594      | 32.89               |
| Weighted Random | 68,153        | 9,902       | 14.53               |
| Worst Bot       | 46,341        | 46,341      | 100.00              |
| Raise Bot       | 52,890        | 0           | 0.00                |
| Cautious        | 110,722       | 22,717      | 20.52               |

Before I made an important fix to *Cautious* while making my statistical analysis earlier this month, it folded 37.59% of the time, which was even more often than *Random Action* did. This fold frequency also confirms that *Random Action* folds as often as it has been defined to.

This next table defines which player won the most overall. Win Rate is a bit inflated since the statistic includes each behavior/bot's wins against *Worst Bot*, which is effectively a free win.

Table 7: Win Count + Win Rate

| Behavior        | Games Played | Wins  | Win Rate |
|-----------------|--------------|-------|----------|
| Random Action   | 8,000        | 2,812 | 0.3515   |
| Weighted Random | 8,000        | 5,646 | 0.7058   |
| Worst Bot       | 8,000        | 31    | 0.0039   |
| Raise Bot       | 8,000        | 6,866 | 0.8583   |
| Cautious        | 8,000        | 4,645 | 0.5806   |

As we can observe, *Raise Bot* won the most out of all the bots. This is followed up by *Weighted Random*. *Cautious* won over half the time if we include *Worst Bot* in its win count. Otherwise, we get a  $\frac{4645-2000}{8000-2000} \approx 44.08\%$  win rate.

For our next table, we get a summary of how many rounds each bot played/won:

Table 8: Round Win Rate Summary

| Behavior        | Rounds Played | Rounds Won | Round Win Rate |
|-----------------|---------------|------------|----------------|
| Random Action   | 45,389        | 23,707     | 0.5223         |
| Weighted Random | 41,081        | 28,205     | 0.6866         |
| Worst Bot       | 52,992        | 6,651      | 0.1255         |
| Raise Bot       | 29,774        | 26,246     | 0.8815         |
| Cautious        | 51,766        | 25,692     | 0.4963         |

This shows quite a few things:

- *Random Action*, despite only winning 35% of games, is able to win over 50% of rounds.
- *Weighted Random* and *Raise Bot* have a similar round win rate and game win rate.
- *Worst Bot* has played the most rounds overall. This is likely because it folds before a raise can be called, forcing it to endure as many rounds as chips it has.
- *Raise Bot* has played the least rounds because it always raises.
- *Cautious* loses more than half the rounds even with its wins against *Worst Bot* included.

For our last statistic, let's see which of the bots that make it to the Showdown win more often in this phase:

Table 9: Showdown Win Rate Summary

| Behavior        | Showdown Wins | Appearances | Showdown Win Rate |
|-----------------|---------------|-------------|-------------------|
| Random Action   | 1,002         | 2,090       | 0.4794            |
| Weighted Random | 2,658         | 5,632       | 0.4719            |
| Raise Bot       | 3,382         | 6,910       | 0.4894            |
| Cautious        | 3,905         | 7,262       | 0.5377            |

We have some more interesting observations:

- *Random Action* shows up to showdown the least.
- *Weighted Random* has the lowest Showdown Win Rate, which means that evaluating exclusively by hole cards doesn't help win more often.

- *Raise Bot* wins showdown the second most, which is just from luck since it doesn't consider its cards.
- *Cautious* not only reaches showdown more often than all the other bots, but it also wins more often. It's the only bot with a  $< 50\%$  showdown win rate. However, a nearly  $54\%$  win rate at showdown isn't much greater than everyone else's win rate.

## 7 Validation and Verification

First I compared the win rate *A\_RandomAction* going against itself. Theoretically, the games should converge to 50/50.

| Metric                  | Value                     |
|-------------------------|---------------------------|
| Total Sample Size       | 1000                      |
| Wins for Player 2       | 937                       |
| Observed Win Rate       | 93.7000%                  |
| 95% Confidence Interval | [0.9201, 0.9513]          |
| P-Value                 | 0.0000                    |
| Conclusion              | Statistical Bias Detected |

Table 10: Baseline Validation: Random vs Random

I think the win rate isn't 50/50 because *A\_RandomAction* folds almost 33% of the time. This means that the second *A\_RandomAction* can only act in 66% of scenarios where the first doesn't fold immediately. Let's try again with *RaiseBot*, a bot that never folds, so we can confirm this:

| Metric                  | Value                                |
|-------------------------|--------------------------------------|
| Total Sample Size       | 1000                                 |
| Wins for Player 2       | 500                                  |
| Observed Win Rate       | 50.0000%                             |
| 95% Confidence Interval | [0.4685, 0.5315]                     |
| P-Value                 | 1.0000                               |
| Conclusion              | Validation Successful ( $p > 0.05$ ) |

Table 11: Baseline Validation: Raise Bot vs Raise Bot

A perfect 50% win rate is greatly exciting to see. This means that *A\_RandomAction* is entirely at fault for performing worse as player 1. Now of course, there is one more issue that I need to address. A bot should logically only fold in response to a raise. After all, there is no gain for folding to a check/call. This is the result of not implementing opponent modeling sooner. Therefore, I'll go on a quick detour to implement it. Here is the easiest way to implement opponent modeling:



When a bot raises, it updates a variable called `proposed_bet`. When the other bot calls or raises, that causes the `current_bet` to equal the `proposed_bet`. Therefore, if `current_bet`  $\geq$  `proposed_bet`, that means that the other bot has checked/called, and `proposed_bet - current_bet = previous_action`.

## 8 Discussion

The results show that without knowledge of the other opponent's actions, never folding gives the player a better chance at winning. You can see that the Win Rate inversely correlates with the fold rate. If you focus on your own cards and are too harsh in evaluation, you will lose a game of attrition by giving your chips away faster than you can win them back in Showdown. In fact, it seems that Cautious doesn't even win Showdown enough of the time to justify that strictness. The simulation reveals that without a more in-depth understanding of your own card potential and your opponent's actions, the most simple strategy is to never fold. Though I had failed in my objective to make something stronger than a bot that only raises, I had succeeded in creating a system that allows for implementing even stronger poker bots.

## 9 Conclusion and Future Work

This whole project was to develop a simulation of Heads Up No Limit Texas Hold'em Poker and to develop a framework under which new A.I behaviors could easily be implemented. I have learned how to develop programs in C++, including C++ programming standards and how to import new libraries. There are many improvements I could make. One would be doing a general check of visibility modifiers to make certain attributes or functions private instead of public. I could modify all behaviors to only fold if the previous action taken was a raise. This would likely improve the performance of every bot. However, given the current date and the stage at which I made opponent modeling even possible, I would then have to redo my entire analysis and every single behavior algorithm. Similarly, I could completely overhaul the A.I. section to call lua scripts instead of using class inheritance to make different bots. This would have the advantage of allowing for new behavior implementation without forking the project entirely. However, this would also require a substantial overhaul to the code, and working with another language that I have not touched before.

If I were to take this information in a different direction, I would probably apply this concept to UNO, an n-person zero-sum card game of imperfect information. Unlike Poker, your goal is to strategically decide which card to play with the end goal of removing all your cards. Decisions made are more subtle and it is even harder to gauge the consequences of the actions made. For instance, players need to decide which turn direction is more beneficial when deciding whether to play a reverse card because the worse decision would result the player being at risk of being affected by another player's aggressive card

play. However, UNO is a game with an even less standardized ruleset than Poker (despite being substantially newer), with many players rejecting the official ruleset in favor of "household" rules. Either way, this project has equipped me with a greater knowledge of simulation logic and game theory, allowing me to make these more ambitious dreams a reality if I wish.

## References

- [1] "Theory Of Games And Economic Behavior: Neumann, John Von. : Free Download, Borrow, and Streaming : Internet Archive," Internet Archive, 2017. <https://archive.org/details/in.ernet.dli.2015.215284/page/n5/mode/2up>
- [2] D. Billings, D. Papp, J. Schaeffer, and D. Szafron, "Opponent Modeling in Poker." Available: <https://cdn.aaai.org/AAAI/1998/AAAI98-070.pdf>
- [3] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret Minimization in Games with Incomplete Information." Accessed: Feb. 07, 2026. [Online]. Available: <https://johanson.ca/publications/poker/2007-nips-cfr/2007-nips-cfr.pdf>
- [4] O. Tammelin, N. Burch, M. Johanson, and M. Bowling, "Solving Heads-up Limit Texas Hold'em." Accessed: Feb. 07, 2026. [Online]. Available: <https://poker.cs.ualberta.ca/publications/2015-ijcai-cfrplus.pdf>
- [5] HenryRLee PokerHandEvaluator: <https://github.com/HenryRLee/PokerHandEvaluator?tab=readme-ov-file>
- [6] SQLite3 Amalgamation: <https://sqlite.org/amalgamation.html>

## 10 Appendix

| Bot A          | Bot B          | WinRate P1 | WinRate P2 | Pos. Adv. | CI Lower | CI Upper | P-Value | Sig |
|----------------|----------------|------------|------------|-----------|----------|----------|---------|-----|
| A_RandomAction | WeightedRandom | 0.116      | 0.170      | -0.054    | -0.085   | -0.023   | 0.0007  | Y   |
| A_RandomAction | WorstBot       | 0.969      | 1.000      | -0.031    | -0.042   | -0.020   | 0.0000  | Y   |
| A_RandomAction | RaiseBot       | 0.082      | 0.095      | -0.013    | -0.038   | 0.012    | 0.3448  | N   |
| A_RandomAction | Cautious       | 0.131      | 0.249      | -0.118    | -0.152   | -0.084   | 0.0000  | Y   |
| WeightedRandom | WorstBot       | 1.000      | 1.000      | 0.000     | 0.000    | 0.000    | 1.0000  | N   |
| WeightedRandom | RaiseBot       | 0.442      | 0.396      | 0.046     | 0.003    | 0.089    | 0.0414  | Y   |
| WeightedRandom | Cautious       | 0.555      | 0.539      | 0.016     | -0.028   | 0.060    | 0.5004  | N   |
| WorstBot       | RaiseBot       | 0.000      | 0.000      | 0.000     | 0.000    | 0.000    | 1.0000  | N   |
| WorstBot       | Cautious       | 0.000      | 0.000      | 0.000     | 0.000    | 0.000    | 1.0000  | N   |
| RaiseBot       | Cautious       | 0.942      | 0.939      | 0.003     | -0.018   | 0.024    | 0.8501  | N   |

Table 12: Turn Order Advantage Analysis (Bot 1 Perspective)